

Uart_Esp32 - Projektdokumentation - Firmware

UART-Konsole und BX3 Image-Transfer | Version v0.17

Projektname	Uart_Esp32
Dokumentenart	Projektdokumentation
Thema	Kopierfreundliche UART-Konsole und verifizierter BX3 UART Image-Transfer
Erstellungsdatum	2026-09-21
Autor / verantwortliche Person	OpenAI ChatGPT (technische Bearbeitung)
Version	v0.17
Status	Entwurf - Syntaxpruefungen erfolgreich; PlatformIO-Build und Hardwaretest ausstehend
Aenderungsdatum	2026-09-21
Ablageort	Projektordner Uart_ESP32/docs
Referenzen	BX3_ESP32_Login_Leitfaden.pdf; Regeln_Projektdokumentation_PDF_DOCX_Pflicht_.pdf; Projektstand v0.16

Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.14	2026-09-20	OpenAI ChatGPT	Entwurf	Bidirektionale BX3-Webkonsole ergaenz.
v0.15	2026-09-20	OpenAI ChatGPT	Entwurf	Gemeinsamer UART-Basisbetrieb, getrennte Menues fuer Einstellungen, Konsole, Decoder und Probe-Runner.
v0.16	2026-09-20	OpenAI ChatGPT	Entwurf	Autoscroll-Haekchenfeld in der UART-Konsole.
v0.17	2026-09-21	OpenAI ChatGPT	Entwurf	Kopierfreundliche Konsole mit Anzeige-Pause und Copy-Funktionen sowie blockweiser, SHA-256-gepruefter BX3 UART Image-Transfer.

1. Ziel der Version v0.17

Version v0.17 erweitert den bestehenden gemeinsamen UART-Basisbetrieb um zwei Punkte:

1. Die UART-Konsole soll auch bei laufendem Live-Empfang verlaesslich markiert und kopiert werden koennen.
2. Ein read-only MTD-Image des BX3 soll ueber die vorhandene serielle Linux-Konsole blockweise zum ESP32 und weiter zum Webbrowser uebertragen werden koennen.

Die bestehende Infrastruktur bleibt erhalten: WLAN/NTP, MQTT, Batterie/ADC, Deep Sleep, OTA, UART Einstellungen, UART Decoder und UART Probe-Runner werden nicht entfernt.

2. Menuestruktur

Die UART-Navigation besteht ab v0.17 aus fuenf Bereichen:

Menue	Aufgabe
UART Einstellungen	Hardware-UART, RX/TX GPIO, Baudrate, Frameformat, Start/Stop und TX-Freigabe.
UART Konsole	Universelles Live-Terminal, manuelle TX-Eingaben und kopierbare Darstellung.
UART Decoder	Bestehender st10-Decoder, Rohdaten und experimenteller Feld-5-Replay.
UART Probe-Runner	Aktiver Test der 1000 Kandidaten.
UART Image-Transfer	Blockweises Backup eines read-only MTD-Devices ueber die BX3-Shell.

3. Gemarkte Aenderung: kopierfreundliche UART-Konsole

3.1 Ursache des bisherigen Problems

Der Browser fragt neue UART-Daten alle 150 ms ab. In v0.16 wurde bei jeder Aktualisierung der komplette sichtbare Terminaltext erneut in `terminal.textContent` geschrieben. Eine Textmarkierung konnte dadurch waehrend des Kopierens verloren gehen.

3.2 Anzeige pausieren

Neu ist das Haekchenfeld **Anzeige pausieren (UART-Empfang laeuft weiter)**.

- Nur die sichtbare Browserdarstellung wird eingefroren.
- UART RX auf dem ESP32 laeuft weiter.
- Neue Daten werden weiterhin in den Browserpuffer uebernommen.
- Nach dem Deaktivieren der Pause wird die Ansicht mit dem aktuellen Pufferstand neu aufgebaut.
- Die Einstellung wird browserseitig in `localStorage` gespeichert.

3.3 Schutz aktiver Textmarkierungen

Die Renderfunktion prueft, ob im Terminal gerade Text markiert ist. Solange eine aktive Auswahl im Terminal besteht, wird `terminal.textContent` nicht ersetzt. Damit funktioniert normales Markieren mit der Maus und anschliessendes `Strg+C` deutlich stabiler.

Fuer das Terminal ist ausserdem explizit `user-select:text` gesetzt.

3.4 Copy-Schaltflaechen

Zusaetzlich stehen zur Verfuegung:

- **Markierten Text kopieren**
- **Gesamte Ansicht kopieren**

Wenn die sichere Clipboard-API des Browsers verfuegbar ist, wird sie verwendet. Bei normalem HTTP versucht die Seite als Fallback einen temporaeren Textbereich und `document.execCommand('copy')`. Unabhaengig davon bleibt normales `Strg+C` moeglich.

3.5 Autoscroll bleibt erhalten

Autoscroll aus v0.16 bleibt unveraendert vorhanden. Anzeige-Pause und Autoscroll sind voneinander getrennt:

Zustand	Verhalten
Autoscroll EIN, Pause AUS	Live-Anzeige folgt dem Datenende.
Autoscroll AUS, Pause AUS	Live-Anzeige aktualisiert sich, Scrollposition wird nicht automatisch ans Ende gesetzt.
Pause EIN	Sichtbarer Inhalt bleibt stehen; UART-Empfang laeuft weiter.
Text markiert	Live-Neuzeichnen wird automatisch unterdrueckt, bis die Auswahl aufgehoben ist.

4. UART Image-Transfer - Zweck und Grenzen

Der neue Bereich `/uart/image` ist fuer ein verifiziertes Backup eines read-only MTD-Devices des BX3 vorgesehen. Standardquelle ist:

```
/dev/mtd7ro
```

Version 1 verwendet bewusst nur bereits auf dem BX3 erwartete Linux-Werkzeuge:

```
dd
base64
sha256sum
wc
cat /proc/mtd
```

Die Firmware schreibt nicht auf das MTD-Device. Als Quelle werden nur Pfade nach dem Muster `/dev/mtdNr` mit `N = 0...31` akzeptiert.

5. Exklusive UART-Reservierung

Waehrend des Image-Transfers besitzt die Image-Funktion den UART exklusiv.

```
UART RX -> Image-Parser
UART TX -> Image-Kommandos
Konsole TX -> gesperrt
Probe-Runner -> gesperrt
Feld-5-Replay -> gesperrt
Decoder -> nicht mit Image-Base64 gespeist
```

Der allgemeine RX-Bytezaehler laeuft weiter. Image-Daten werden jedoch bewusst nicht in den normalen Konsolen-, Rohdaten- oder Decoderpfad eingestreut.

txOwnerText() meldet waehrenddessen UART Image-Transfer.

6. Startbedingungen und Groessenermittlung

Der Transfer startet nur, wenn:

- UART laeuft,
- TX unter UART Einstellungen explizit freigegeben ist,
- ein gueltiger TX-GPIO vorhanden ist,
- kein Probe-Runner aktiv ist,
- kein Feld-5-Replay aktiv ist,
- kein anderer Image-Transfer laeuft,
- die Quelle dem Muster /dev/mtdNro entspricht.

Vor dem ersten Block sendet die Firmware einen eindeutigen <<<BX3IMG:MTD_BEGIN>>>-Marker und nutzt dessen Rueckgabe als Shell-Synchronisierung. Danach liest sie fuer die gewaehlte MTD-Nummer /proc/mtd. Die hexadezimale Partitionsgroesse wird daraus ermittelt. Fuer ein 16-MiB-Device ergibt sich bei 32 KiB Blockgroesse:

```
16 MiB / 32 KiB = 512 Bloecke
```

7. Blockweises Request/Response-Protokoll

Die Blockgroesse ist in v0.17 fest auf **32768 Byte** gesetzt.

Fuer jeden Block erzeugt der ESP32 einen Shell-Befehl, der den angeforderten Ausschnitt nach /tmp/bx3blk liest und anschliessend Groesse, SHA-256 und Base64-Daten ausgibt.

Die Uebertragung verwendet eindeutige Marker:

```
<<<BX3IMG:BEGIN:000017>>>
<<<BX3IMG:SIZE:32768>>>
<<<BX3IMG:SHA256:abcd...>>>
<Base64-Daten>
<<<BX3IMG:END:000017>>>
```

Der naechste Block wird erst angefordert, nachdem der aktuelle Block vollstaendig validiert und vom Browser bestaetigt wurde.

8. Image-Parser

Die Firmware verwendet einen eigenen Zustandsautomaten:

```
WaitMtd
-> WaitBegin
-> WaitSize
-> WaitSha
-> RxBase64
-> BlockReady
-> naechster Block
-> WaitTotalSha
-> Done
```

Fehler fuehren zu einem Retry oder zu Error.

Der Base64-Text wird zeilenweise dekodiert. Der ESP32 haelt dabei nur den aktuell verarbeiteten 32-KiB-Block im statischen Blockpuffer. Das vollstaendige Image wird nicht im ESP32-RAM gespeichert.

9. Integritaetspruefung pro Block

Jeder Block wird zweistufig geprueft:

1. Die dekodierte Laenge muss exakt der erwarteten Blocklaenge entsprechen.
2. Der lokal auf dem ESP32 berechnete SHA-256 muss dem vom BX3 gelieferten SHA-256 entsprechen.

Bei Fehler wird derselbe Block erneut angefordert. Pro Block sind maximal **5 Wiederholungen** vorgesehen. Danach stoppt der Transfer mit Fehlerstatus.

Beispiele fuer Fehlerursachen:

- UART Timeout,
- unguelteiger SIZE-Marker,
- unguelteiger SHA256-Marker,
- Base64-Fehler,
- Blockpufferueberlauf,
- falsche dekodierte Laenge,
- SHA-256-Abweichung.

10. Gesamt-SHA256

Bei einem Volltransfer ab Startblock 0 aktualisiert der ESP32 waehrend jedes Browser-ACKs parallel einen Gesamt-SHA256 ueber alle validierten Binärbloেকে.

Nach dem letzten Block fordert der ESP32 vom BX3 zusaetzlich an:

```
sha256sum /dev/mtd7ro
```

Der lokale und der entfernte Hash muessen uebereinstimmen. Nur dann lautet der Endstatus:

```
IMAGE VERIFIED
```

Bei einem Startblock groesser als 0 kann der ESP32 keinen SHA-256 ueber den fehlenden Anfang des Images berechnen. Der Teiltransfer bleibt blockweise geprueft, wird aber nicht als vollstaendig gesamtverifiziert bezeichnet.

11. Browser-Download

Die ESP32-Weboberflaeche sammelt die validierten Bloেকে im Browser, nicht im ESP32.

Ablauf:

```
BX3 dd/base64
-> ESP32 Image-Parser
-> Block-SHA256 OK
-> GET /uart/image/block
-> Browser ArrayBuffer
-> POST /uart/image/ack
-> naechster Block
```

/uart/image/block liefert den bereits validierten Block als application/octet-stream ueber eine chunked Response. Dadurch muss der ESP32 fuer die Browserantwort keine zweite Base64-Kopie des 32-KiB-Blocks erzeugen.

Nach dem letzten verifizierten Block erzeugt der Browser aus den gesammelten Uint8Array-Bloেকে einen Blob und stellt die Datei als Download bereit, beispielsweise:

```
bx3-mtd7.img
```

Bei einem Startblock groesser als 0 wird ein Teilimage mit entsprechendem Dateinamen angeboten.

Abweichung vom urspruenglichen Streaming-Entwurf: v0.17 streamt nicht eine einzige stundenlange HTTP-Verbindung vom ersten bis zum letzten UART-Block. Stattdessen wird jeder validierte Block separat an den Browser uebertragen und anschliessend bestaetigt. Das passt besser zum asynchronen Webserver und macht einen verlorenen HTTP-Block erneut abrufbar, ohne den BX3-Block sofort weiterzuschalten.

12. Resume-Funktion

Die Weboberflaeche besitzt ein Feld **Startblock**. Damit kann der UART-Transfer gezielt an einem spaeteren Flashoffset beginnen.

Beispiel:

```
Startblock: 237
```

Das erzeugt in v0.17 ein Teilimage ab Block 237. Es ist noch kein transparentes HTTP-Range-Resume einer bereits lokal vorhandenen Datei. Fuer ein vollstaendiges Image muessen Teilstuecke ausserhalb des ESP32 korrekt zusammengesetzt werden.

13. Abbruch und Fehlerbehandlung

Bei Abbruch oder nicht wiederherstellbarem Fehler:

- wird /tmp/bx3blk auf dem BX3 geloescht,
- wird der Image-Zustand auf Fehler gesetzt,
- wird die SHA-256-Berechnung freigegeben,
- endet die exklusive Image-Reservierung,
- Konsole, Probe-Runner und sonstige TX-Funktionen koennen danach wieder verwendet werden.

Auch UART STOP beendet einen aktiven Image-Transfer sauber, bevor HardwareSerial geschlossen wird.

14. Webrouen v0.17

Route	Funktion
/uart/image	Image-Transfer-Webseite.
/uart/image/status	JSON-Livestatus mit Block, Groesse, Fehlern, Retries, Rate und Hashes.
/uart/image/start	Startet Transfer fuer ein read-only MTD-Device und optionalen Startblock.
/uart/image/block	Liefert den aktuell validierten Binärblock als application/octet-stream.
/uart/image/ack	Bestaetigt den Browserempfang und fordert danach den naechsten Block an.
/uart/image/abort	Bricht den Transfer ab und fuehrt Cleanup aus.

Die bisherigen Routen fuer Einstellungen, Konsole, Decoder und Probe-Runner bleiben bestehen.

15. Sicherheits- und Betriebsaspekte

- Der Transfer setzt eine bereits autorisierte BX3-Shell voraus. Die Firmware umgeht keine Zugangsdaten.
- Das Webinterface verwendet weiterhin normales HTTP. Image- und Konsolenfunktionen sollen nur in einem vertrauenswuerdigen Netz verwendet werden.
- TX muss bewusst freigegeben sein.
- Die im BX3-Leitfaden beschriebenen elektrischen Regeln bleiben bestehen: gemeinsame Masse, Logikpegel pruefen, VCC nicht verbinden und TX bei der Erprobung ueber einen Schutzwiderstand anschliessen.
- Die Image-Funktion akzeptiert absichtlich nur read-only MTD-Geraete /dev/mtdNro.

16. Geaenderte Dateien

Datei	Aenderung
src/UartManager.h	Image-Zustaende, 32-KiB-Blockpuffer, SHA-Zustand und

	Image-API ergaenzt.
src/UartManager.cpp	Exklusive Image-UART-Verarbeitung, MTD-Groessenermittlung, Markerparser, Base64-Dekodierung, Block- und Gesamt-SHA256, Retry/Abort sowie TX-Sperren implementiert.
src/main.cpp	Firmwarebuild v0.17, Image-Menue und Webrouen, Browser-Blockdownload sowie kopierfreundliche Konsolenoberflaeche.
src/ConfigDefaults.h	Firmwareversion auf 0.17.0 angehoben.
README.md	Neuer Stand v0.17 und neue Funktionen beschrieben.
VALIDATION_v0.17.txt	Validierungsstand der Aenderung.
docs/2026-09-21 - Uart_Esp32 - Projektdokumentation - Firmware - v0.17.*	PDF, DOCX und Markdown dieser Dokumentation.
docs/2026-09-21 - Uart_Esp32 - Commit-Nachricht - UART-Image-Transfer-und-Konsole - v0.17.txt	Englische Commit-Nachricht nach Projektrichtlinie.

17. Validierung

Durchgefuehrt wurden:

- lokale C++17-Syntaxpruefung von UartManager.cpp und TestCandidateCatalog.cpp gegen ESP32/Arduino-Host-Stubs,
- lokale C++17-Syntaxpruefung von main.cpp gegen Webserver-Host-Stubs,
- JavaScript-Syntaxpruefung aller aus den C++-Raw-Literalen extrahierten Webskripte mit Node.js,
- statische Kontrolle der fuenf UART-Menues und neuen Image-Routen,
- Kontrolle, dass Image-Transfer in consoleTxReady(), Feld-5-Replay und Probe-Runner als TX-Sperre beruecksichtigt wird,
- ZIP- und Patch-Pruefung bei der Auslieferung,
- visuelle Renderpruefung der DOCX- und PDF-Dokumentation.

Nicht durchgefuehrt werden konnten:

- echter PlatformIO-Build mit der ESP32-Arduino-Toolchain,
- Flashen auf ein ESP32-WROOM-32 / NodeMCU-32S,
- Hardwaretest an einem realen BX3,
- Langzeittest eines 16-MiB-Transfers ueber WLAN und UART.

Vor dem Flashen ist deshalb mindestens auszufuehren:

```
pio run -e uart_esp32_usb
```

18. Inbetriebnahme-Checkliste v0.17

- ☐ UART-Einstellungen und GPIOs kontrollieren.
- ☐ RX-only zuerst pruefen und BX3-Ausgabe lesen.
- ☐ Logikpegel und gemeinsame Masse pruefen.
- ☐ TX erst danach bewusst freigeben.
- ☐ UART-Konsole testen; Markieren, Anzeige-Pause und Copy-Funktionen pruefen.
- ☐ Vor dem Image-Transfer sicherstellen, dass eine autorisierte BX3-Shell erreichbar ist.
- ☐ /dev/mtd7ro bzw. die gewuenschte read-only MTD-Quelle auf dem BX3 verifizieren.
- ☐ Image-Transfer zuerst mit kleinem/unkritischem Ziel oder Testsystem pruefen.
- ☐ Fuer ein Vollbackup Startblock 0 verwenden.
- ☐ Nach Abschluss IMAGE VERIFIED sowie identische lokale und BX3-SHA256 kontrollieren.
- ☐ Erzeugte Image-Datei zusaetzlich auf dem PC hashen und sicher archivieren.

19. Offene Punkte fuer eine spaetere Version

- Echtes Binärprotokoll statt Base64 zur Reduzierung des Overheads.
- Optionaler SD-Karten-Zwischenspeicher fuer robuste Wiederaufnahme nach WLAN-Abbruch.
- Vollstaendiges HTTP-Range-/Resume-Konzept mit Zusammenfuehrung bereits vorhandener lokaler Teilimages.
- Hardwaremessung, ob das BX3 fuer Image-Transfers stabil mit hoeheren Baudraten als 115200 arbeitet.
- Optional WebSocket fuer die normale UART-Livekonsole.